



Universidad Nacional de Colombia

FACULTAD DE CIENCIAS

LABORATORIO 1

Advanced Data Structures
Prof. Juan Carlos Mendiavelso Moreno

Presentado por:

Angel David Gomez Pastrana

Bogotá, Mayo del 2026

1. Introducción

El problema de *Range Minimum Query* (RMQ) es un problema fundamental en el área de estructuras de datos y algoritmos. Dado un arreglo A de n elementos, el objetivo consiste en responder eficientemente a consultas sobre el valor mínimo contenido dentro de un rango específico del arreglo.

Formalmente, para dos índices i y j tales que $0 \leq i \leq j < n$, una consulta RMQ se define como:

$$RMQ(i, j) = \min_{i \leq k \leq j} A[k]$$

o, dependiendo de la implementación, como el índice en el que se encuentra dicho mínimo.

El problema RMQ aparece en numerosas aplicaciones prácticas, incluyendo sistemas de bases de datos, procesamiento de texto, bioinformática, análisis geoespacial y algoritmos sobre árboles. Además, mantiene una estrecha relación con el problema de *Lowest Common Ancestor* (LCA), uno de los problemas clásicos en teoría de grafos y estructuras jerárquicas.

La solución más simple consiste en recorrer todos los elementos comprendidos entre las posiciones i y j , obteniendo una complejidad temporal de $O(j - i + 1)$ por consulta. Sin embargo, cuando se requiere responder a una gran cantidad de consultas sobre un mismo arreglo, esta estrategia resulta ineficiente. Por esta razón, se han desarrollado diversas estructuras de datos que realizan un preprocesamiento inicial para acelerar significativamente el tiempo de respuesta de las consultas.

Entre las estructuras más conocidas para resolver RMQ se encuentran:

- **Naive RMQ:** no realiza preprocesamiento y responde cada consulta mediante un recorrido lineal del rango.
- **Block Decomposition:** realiza un preprocesamiento basado en una división por bloques que ofrece un rendimiento teórico de $O(n)$ en preprocesamiento y $O(n/b)$ en consultas, donde b representa el tamaño de los bloques.
- **Sparse Table:** utiliza un preprocesamiento de complejidad $O(n \log n)$ para responder consultas en tiempo $O(1)$.
- **Segment Tree:** ofrece un equilibrio entre memoria y rendimiento, con preprocesamiento $O(n)$ y consultas $O(\log n)$.
- **Fischer–Heun:** estructura teóricamente óptima que alcanza preprocesamiento $O(n)$, espacio $O(n)$ y consultas $O(1)$.

El objetivo de este trabajo es comparar experimentalmente estas y otras implementaciones de estructuras de datos para RMQ en términos de tiempo de preprocesamiento, tiempo de consulta, consumo de memoria y escalabilidad. A través de este análisis, es posible identificar las ventajas y desventajas de cada estructura (*trade-offs*) y determinar cuáles son más adecuadas según los requerimientos de una aplicación específica.

2. Planteamiento del experimento

Para iniciar, es importante mencionar algunas consideraciones sobre el desarrollo del experimento.

El experimento fue desarrollado en *Java*, principalmente por cuestiones de eficiencia. Aunque *Java* no posee el mismo rendimiento que lenguajes como *C* o *C++*, sigue ofreciendo un desempeño competitivo y cuenta con ventajas adicionales, como el *Garbage Collector*, que facilita la gestión de memoria, así como un sólido sistema de clases e interfaces que simplifica la definición y organización del experimento.

Pasando a aspectos más concretos, definiremos qué se va a medir, cómo se realizará la medición y bajo qué condiciones se desarrollará el experimento.

2.1. Diseño del experimento

Queremos obtener una idea de cómo se comporta cada estructura de datos para distintos tamaños de entrada en términos de tiempo de preprocesamiento, tiempo de consulta y uso de memoria. Para ello se utilizarán arreglos de tamaños 2^{10} , 2^{12} , 2^{14} , 2^{16} , 2^{18} y 2^{20} . Sin embargo, en el caso de la implementación de *Full Preprocessing* (explicada más adelante), únicamente se realizarán pruebas hasta 2^{14} debido a restricciones de memoria.

Se realizará una fase de validación de correctitud de las implementaciones. Para ello se generará un arreglo aleatorio de tamaño 5000 con valores entre 0 y 10^9 , junto con 10000 consultas aleatorias. Los resultados obtenidos por cada estructura serán comparados contra los producidos por la implementación *Naive RMQ*. En caso de encontrarse alguna discrepancia, el experimento se abortará y se reportarán los errores encontrados.

Posteriormente, para la recolección de datos experimentales, se generarán arreglos de números aleatorios uniformemente distribuidos entre 0 y 10^9 . Se utilizará una semilla fija para garantizar la reproducibilidad del experimento.

Para cada arreglo se generará una lista de 5 000 000 consultas cuando el tamaño del arreglo sea menor o igual a 2^{14} y de 1 000 000 consultas en caso contrario. Estas consultas se ejecutarán secuencialmente y se medirá el tiempo requerido para completarlas.

Cada experimento será repetido 30 veces. En cada repetición la estructura deberá construirse desde cero. Finalmente, se reportarán las estadísticas requeridas sobre el tiempo total de ejecución.

Para la fase de preprocesamiento, se medirá el tiempo total requerido para construir las estructuras auxiliares necesarias (tablas, árboles, arreglos auxiliares, etc.) y se reportarán las estadísticas correspondientes sobre las 30 ejecuciones.

En cuanto al uso de memoria, cada implementación deberá calcular explícitamente la cantidad de bytes utilizados por la estructura de datos construida.

2.2. Implementación del experimento

Con el objetivo de hacer el experimento más claro y sencillo de mantener, el constructor de cada clase se utiliza únicamente para inicializar las estructuras necesarias (variables, arreglos,

matrices, listas, etc.) y colocar el arreglo que se usara para el procesamiento en la estructura. En esta etapa no se contabilizan tiempos de preprocesamiento ni uso de memoria. En algunas estructuras se requiere el uso de la función $\log_2(n)$ para distintos cálculos; *Java* no posee una función explícita que calcule $\log_2(n)$, por lo tanto, usamos una tabla para guardar algunos valores que se usaran en la estructura y hacer que esta operación cumpla con ser constante. Esta tabla no se contabiliza dentro del costo de memoria dentro de la estructura de datos, dado que se supone que es una operación y no una tabla auxiliar.

Posteriormente se definió una interfaz común que todas las implementaciones deben extender. Esta interfaz contiene cuatro métodos que permiten evaluar todas las estructuras bajo las mismas condiciones:

- **void preprocess():** este método llena las partes importantes de la estructura de datos (tablas auxiliares, variables auxiliares y la propia estructura de datos). Este es el método utilizado para medir el tiempo de preprocesamiento bajo igualdad de condiciones.
- **int RMQ(int i, int j):** método principal encargado de resolver consultas RMQ. Para este experimento devuelve el índice de la posición del elemento mínimo comprendido entre i y j .
- **void countMemoryBytes():** calcula la cantidad de bytes utilizados por la implementación y almacena el resultado en una variable privada para preservar la integridad del experimento.
- **long getMemoryBytes():** devuelve la cantidad de bytes utilizada por la implementación.

Además, para facilitar el desarrollo y evitar código duplicado, se creó el directorio `./src/utills`, el cual contiene cinco clases auxiliares:

- **CSVHandler.java:** auxilia toda la parte de escritura de los datos en archivos *CSV*. De esta forma es posible centralizar la lógica de exportación y simplificar el código del experimento.
- **StatsUtils.java:** auxilia en el cálculo de estadísticas para distintos tipos de variables.
- **RandomGenerator.java:** se encarga de generar tanto las consultas como los arreglos aleatorios, con o sin semilla.
- **Validator.java:** auxilia en la validación de correctitud del experimento. Contiene una implementación del algoritmo *Naive RMQ* para verificar las consultas realizadas por las distintas estructuras de datos.
- **ExperimentDataStructure.java:** su método principal, `Experiment()`, se encarga de evaluar la estructura de datos, tomar las mediciones requeridas y almacenarlas en variables privadas accesibles únicamente mediante métodos *getter*. Además, incorpora verificaciones mediante la clase `Validator` para asegurar la correctitud de todas las consultas ejecutadas.

Todo el proceso es orquestado desde `Main.java`, encargado de ejecutar los experimentos y controlar el flujo general del programa. Adicionalmente, se creó un archivo `setup.bat` para automatizar la compilación y ejecución de `Main.java`. Todos los datos tomados se guardan en el directorio `./csv`.

Para la generación de las gráficas se utilizó un conjunto de scripts en *Python*, ubicados en el directorio `./plots`. El archivo principal es `generate_plots.py`, el cual utiliza funciones definidas en `plot_functions.py` para simplificar la creación de las visualizaciones. Todas las gráficas generadas se almacenan en el directorio `./plots/images`.

3. Implementaciones.

A continuación se presenta un resumen de las estructuras de datos implementadas para resolver el problema RMQ, junto con la idea principal detrás de cada una de ellas.

3.1. Naive RMQ

La implementación más sencilla consiste en almacenar únicamente el arreglo original y responder cada consulta recorriendo linealmente todos los elementos comprendidos entre los índices consultados. No requiere ningún tipo de preprocesamiento adicional, por lo que su costo de construcción es prácticamente nulo. Sin embargo, el tiempo de consulta crece linealmente con el tamaño del intervalo analizado.

- Preprocesamiento: $O(1)$
- Consulta: $O(n)$
- Espacio adicional: $O(1)$

3.2. Full Preprocessing

Esta estrategia realiza un preprocesamiento completo de todos los posibles intervalos del arreglo. Para cada par de índices (i, j) se almacena el índice correspondiente al elemento mínimo del rango. Como consecuencia, las consultas pueden responderse en tiempo constante mediante un acceso directo a la tabla precomputada.

La principal desventaja de este enfoque es su elevado consumo de memoria y tiempo de construcción, ambos cuadráticos respecto al tamaño de la entrada.

- Preprocesamiento: $O(n^2)$
- Consulta: $O(1)$
- Espacio adicional: $O(n^2)$

3.3. Sparse Table

La estructura *Sparse Table* se basa en precomputar los mínimos de intervalos cuyos tamaños son potencias de dos. Durante una consulta se combinan dos intervalos superpuestos que cubren completamente el rango solicitado, permitiendo responder en tiempo constante.

Es una de las soluciones clásicas para RMQ estático debido a su excelente rendimiento en consultas.

- Preprocesamiento: $O(n \log n)$
- Consulta: $O(1)$
- Espacio adicional: $O(n \log n)$

3.4. Block Decomposition

También conocida como *Square Root Decomposition*, esta técnica divide el arreglo en bloques de tamaño aproximado $\sqrt{n}$. Para cada bloque se almacena el mínimo correspondiente.

Las consultas se responden recorriendo explícitamente los bloques parciales de los extremos y utilizando los mínimos precomputados para los bloques completos contenidos en el rango.

- Preprocesamiento: $O(n)$
- Consulta: $O(\sqrt{n})$
- Espacio adicional: $O(\sqrt{n})$

3.5. Hybrid A

Esta estructura divide el arreglo en bloques de tamaño $b = \lceil \log_2 n \rceil$. Sobre los mínimos de cada bloque se construye una *Sparse Table*, mientras que los elementos pertenecientes a los bloques parciales de los extremos son procesados mediante búsqueda lineal.

La idea principal consiste en aprovechar la rapidez de una *Sparse Table* para bloques completos y mantener un costo de construcción lineal.

- Preprocesamiento: $O(n)$
- Consulta: $O(\log n)$
- Espacio adicional: $O(n)$

3.6. Hybrid B

Esta variante mejora la estrategia anterior construyendo una *Sparse Table* independiente dentro de cada bloque, además de una segunda *Sparse Table* sobre los mínimos globales de los bloques.

Gracias a esta doble jerarquía, las consultas pueden resolverse en tiempo constante, aunque a costa de un mayor uso de memoria.

- Preprocesamiento: $O(n \log \log n)$
- Consulta: $O(1)$
- Espacio adicional: $O(n \log \log n)$

3.7. Hybrid C

Esta estructura utiliza una *Sparse Table* sobre los mínimos de bloques de tamaño $b = \lceil \log_2 n \rceil$, pero reemplaza las búsquedas lineales internas por una versión recursiva de la estructura *Hybrid A*.

El objetivo es reducir el costo de las consultas sobre los bloques parciales manteniendo un consumo de memoria cercano al lineal.

- Preprocesamiento: $O(n)$
- Consulta: $O(\log \log n)$
- Espacio adicional: $O(n)$

3.8. Fischer-Heun

La estructura de Fischer-Heun es considerada una de las soluciones teóricamente óptimas para RMQ estático. Combina descomposición por bloques, árboles cartesianos, tipos canónicos de bloques y una *Sparse Table* construida sobre resúmenes de bloques.

Gracias a esta combinación logra responder consultas en tiempo constante utilizando únicamente espacio lineal.

- Preprocesamiento: $O(n)$
- Consulta: $O(1)$
- Espacio adicional: $O(n)$

3.9. Segment Tree

El *Segment Tree* es una de las estructuras de datos más utilizadas para consultas sobre intervalos. Se basa en un árbol binario donde cada nodo almacena información agregada sobre un segmento del arreglo.

Aunque no alcanza consultas en tiempo constante, ofrece una buena combinación entre simplicidad, uso de memoria y rendimiento, además de ser fácilmente adaptable a escenarios dinámicos.

- Preprocesamiento: $O(n)$
- Consulta: $O(\log n)$
- Espacio adicional: $O(n)$

4. Análisis de resultados

4.1. Uso de memoria

Para la parte de uso de memoria de las estructuras de datos, los resultados obtenidos muestran un comportamiento consistente con las complejidades teóricas esperadas. Aunque los valores medidos no coinciden exactamente con las expresiones asintóticas, las tendencias observadas reflejan claramente las diferencias de crecimiento entre las distintas implementaciones.

En la Figura 1 se presenta el consumo de memoria en función del tamaño del arreglo utilizando escalas logarítmicas en ambos ejes. Se observa que **Block Decomposition**, cuya complejidad espacial teórica es $O(\sqrt{n})$, presenta la pendiente más baja de todas las estructuras evaluadas, convirtiéndose en la alternativa más eficiente en términos de uso de memoria entre aquellas que requieren estructuras auxiliares.

Sin embargo, el caso más notable es **Naive RMQ**. Como esta implementación no realiza ningún tipo de preprocesamiento ni almacena información adicional aparte de una referencia al arreglo original, su consumo de memoria adicional es esencialmente nulo. Desde el punto de vista espacial, constituye la implementación más económica posible, aunque este beneficio se obtiene a costa de un tiempo de consulta lineal.

A continuación se encuentran las estructuras **Hybrid A** y **Hybrid C**, las cuales muestran prácticamente el mismo consumo de memoria (véase `./csv/memory_usage.csv`). Este comportamiento es consistente con sus complejidades espaciales teóricas cercanas a $O(n)$ y con las similitudes existentes en sus diseños.

Resulta particularmente interesante el caso de **Hybrid B**. A pesar de poseer una complejidad teórica de $O(n \log \log n)$, su consumo de memoria experimental se mantiene relativamente cercano al de **Segment Tree**, cuya complejidad espacial es $O(n)$. Asimismo, la implementación de **Fischer-Heun**, también con complejidad espacial teórica $O(n)$, presenta un consumo de memoria superior al de ambas estructuras, lo que sugiere que las constantes ocultas asociadas a sus estructuras auxiliares tienen un impacto significativo en la práctica.

Por encima de estas estructuras se encuentra **Sparse Table**, cuya pendiente es visiblemente más pronunciada. Este comportamiento es consistente con su complejidad espacial de $O(n \log n)$, lo que provoca un crecimiento más acelerado del uso de memoria conforme aumenta el tamaño de la entrada.

Finalmente, la estructura **Full Preprocessing** exhibe con diferencia el mayor consumo de memoria de todas las implementaciones evaluadas. Su crecimiento cuadrático, $O(n^2)$, se refleja claramente en la gráfica, donde su curva se encuentra completamente separada del resto de las estructuras. Este resultado confirma que, aunque ofrece consultas en tiempo constante, su costo espacial la convierte en una alternativa poco práctica para tamaños de entrada moderados o grandes. De hecho, incluso para tamaños relativamente pequeños como 2^{10} o 2^{12} , el costo espacial comienza a ser considerable frente al resto de alternativas evaluadas.

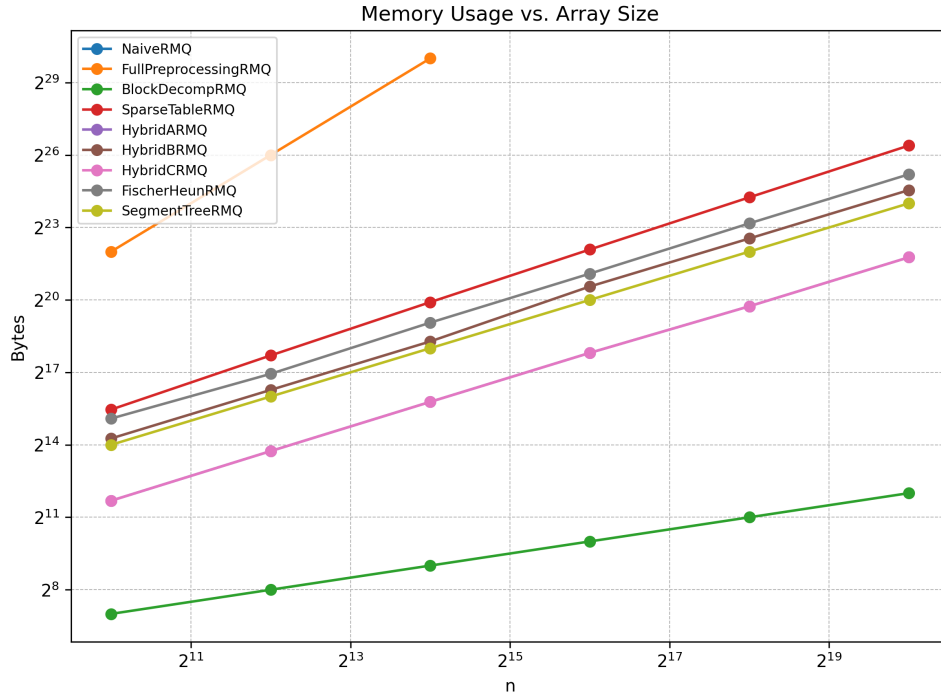


Figura 1: Uso de memoria en función del tamaño del arreglo para las distintas estructuras de datos.

4.2. Preprocesamiento

La Figura 2 muestra el comportamiento del tiempo de preprocesamiento para las distintas estructuras de datos evaluadas.

Como era de esperarse, la implementación **Full Preprocessing** se encuentra considerablemente separada del resto de las estructuras. Su crecimiento cuadrático provoca que el tiempo de construcción aumente rápidamente conforme crece el tamaño de la entrada, convirtiéndola en la estructura más costosa en términos de preprocesamiento. Este comportamiento resulta consistente con los resultados observados anteriormente en el análisis de memoria, donde también fue la estructura con el mayor consumo de recursos.

Por otro lado, la estructura **Fischer-Heun** presenta un comportamiento interesante. Para tamaños pequeños y medianos su tiempo de preprocesamiento permanece relativamente bajo, pero a partir de 2^{14} comienza a incrementarse de manera considerable. Aunque mantiene la complejidad teórica lineal esperada, las constantes ocultas asociadas a su construcción parecen tener un impacto significativo en la práctica.

Resulta llamativo observar que estructuras como **Segment Tree**, **Hybrid A**, **Hybrid C** y **Block Decomposition** presentan tiempos de preprocesamiento menores que los observados para **Fischer-Heun**. Incluso estructuras con complejidades teóricas superiores, como **Sparse Table** ($O(n \log n)$) y **Hybrid B** ($O(n \log \log n)$), presentan tiempos de construcción inferiores a los observados para **Fischer-Heun** en los tamaños evaluados. Esto evidencia nuevamente la importancia de las constantes ocultas y de los detalles de implementación en el rendimiento práctico.

Entre todas las implementaciones, **Block Decomposition** y **Hybrid A** destacan por mantener tiempos de preprocesamiento extremadamente bajos incluso para arreglos de tamaño 2^{20} . En particular, **Hybrid A** resulta especialmente interesante debido a que combina un costo de cons-

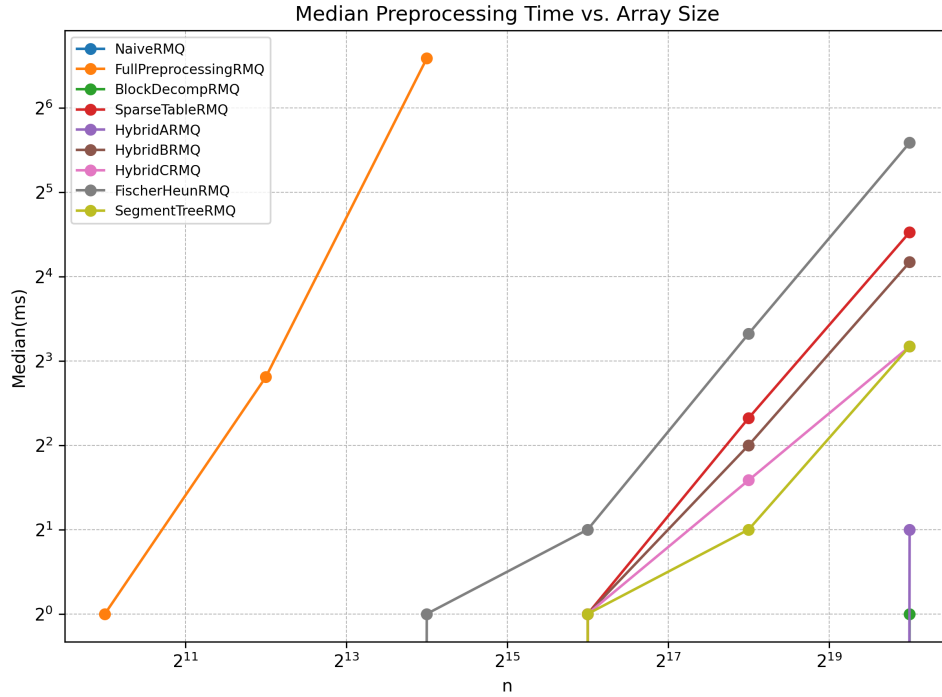


Figura 2: Mediana del tiempo de preprocesamiento en función del tamaño del arreglo para las distintas estructuras de datos.

trucción reducido con un buen desempeño en consultas, como se analizará en la siguiente subsección.

Finalmente, la implementación **Naive RMQ** presenta el menor tiempo de preprocesamiento posible, ya que no realiza ninguna construcción adicional sobre el arreglo original. Junto con **Block Decomposition**, la cual constituye la segunda estructura más rápida de inicializar y primera que tenga algún tipo de preprocesamiento.

4.3. Tiempo de consulta

La Figura 3 presenta la mediana del tiempo de consulta para las distintas estructuras de datos evaluadas.

Los resultados obtenidos muestran diferencias mucho más marcadas entre las estructuras que las observadas durante la fase de preprocesamiento. Como era esperado, **Naive RMQ** presenta el peor desempeño debido a que cada consulta requiere recorrer linealmente el intervalo solicitado. A medida que aumenta el tamaño del arreglo, el tiempo de consulta crece rápidamente, alejándose del resto de las implementaciones.

Por el contrario, las estructuras diseñadas específicamente para optimizar consultas muestran tiempos considerablemente menores. Entre ellas, **Sparse Table** se posiciona consistentemente como la alternativa más rápida, confirmando que su elevado costo de preprocesamiento y memoria se traduce en un excelente rendimiento durante la fase de consultas.

También resulta destacable el comportamiento de **Hybrid A**, que a pesar de poseer una complejidad teórica de consulta $O(\log n)$, logra tiempos muy competitivos e incluso comparables con algunas estructuras teóricamente más eficientes. Esto sugiere que factores prácticos como

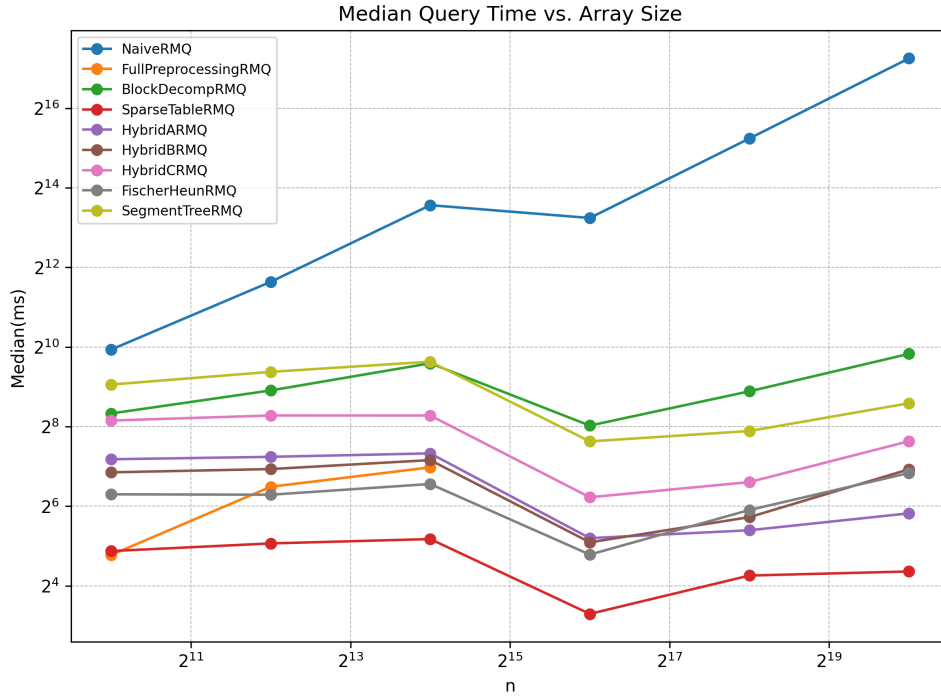


Figura 3: Mediana del tiempo de consulta en función del tamaño del arreglo.

la localidad de memoria, el tamaño reducido de las estructuras auxiliares y menores constantes ocultas tienen un papel importante en el rendimiento real.

Por otro lado, **Fischer-Heun** presenta tiempos de consulta consistentes con una estructura de complejidad teórica $O(1)$. Sin embargo, en términos prácticos no logra superar a **Sparse Table** y en varios casos tampoco a **Hybrid A**. Estos resultados muestran que una mejor complejidad asintótica no garantiza necesariamente un mejor rendimiento experimental. Factores como las constantes ocultas, la localidad de memoria y el comportamiento de la caché pueden compensar diferencias teóricas importantes.

Otro fenómeno observable en la gráfica es la reducción aparente del tiempo de consulta para algunas estructuras cuando se pasa de los tamaños pequeños a los tamaños más grandes. Este comportamiento se explica principalmente porque el número total de consultas ejecutadas cambia durante el experimento: para tamaños hasta 2^{14} se ejecutan 5 000 000 consultas, mientras que para tamaños superiores se ejecutan 1 000 000. Esta diferencia puede introducir variaciones en la medición relacionadas con efectos de caché, optimizaciones del compilador JIT y comportamiento general del sistema durante la ejecución.

4.4. Estabilidad y variabilidad

Los gráficos de estabilidad muestran que, en general, las estructuras evaluadas presentan una diferencia reducida entre la media y la mediana de los tiempos de consulta. Esto indica que la mayoría de las mediciones se encuentran concentradas alrededor de un valor representativo y que los tiempos de ejecución son relativamente consistentes entre repeticiones.

Sin embargo, algunas estructuras exhiben patrones interesantes. En particular, **Hybrid A**, **Hybrid B**, **Sparse Table** y **Fischer-Heun** muestran medias sistemáticamente superiores a sus medianas para ciertos tamaños de entrada. Este comportamiento suele indicar la presencia de valores

atípicos correspondientes a ejecuciones ocasionalmente más lentas.

Para **Fischer-Heun** se observa una separación moderada entre media y mediana, acompañada de una desviación estándar relativamente elevada junto a mínimo y máximo mas extremos. Esto sugiere una mayor sensibilidad a factores externos del entorno de ejecución o localidad de la implementación.

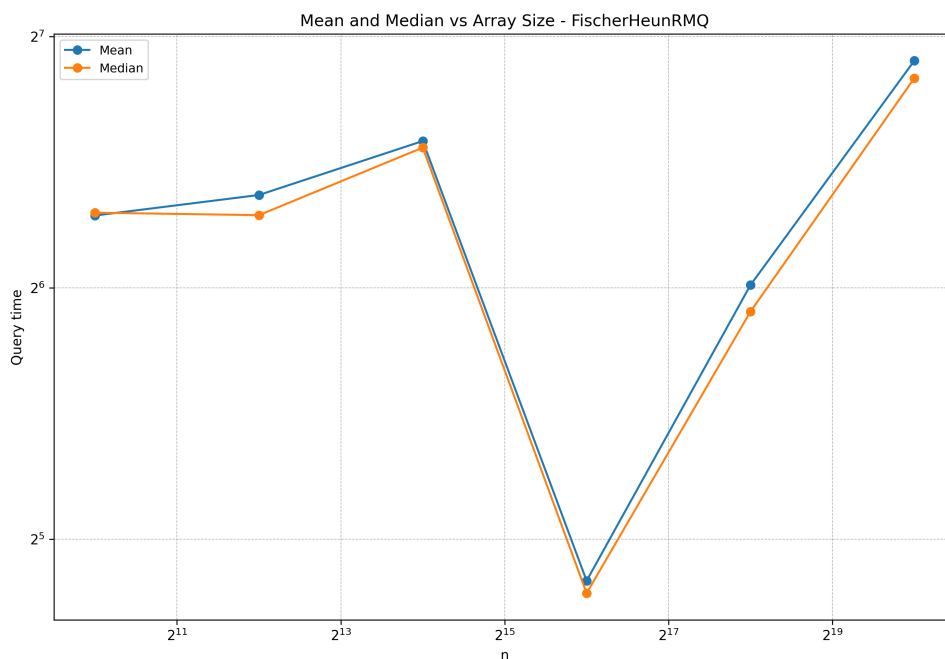


Figura 4: Media y mediana del tiempo de consulta para **Fischer-Heun**.

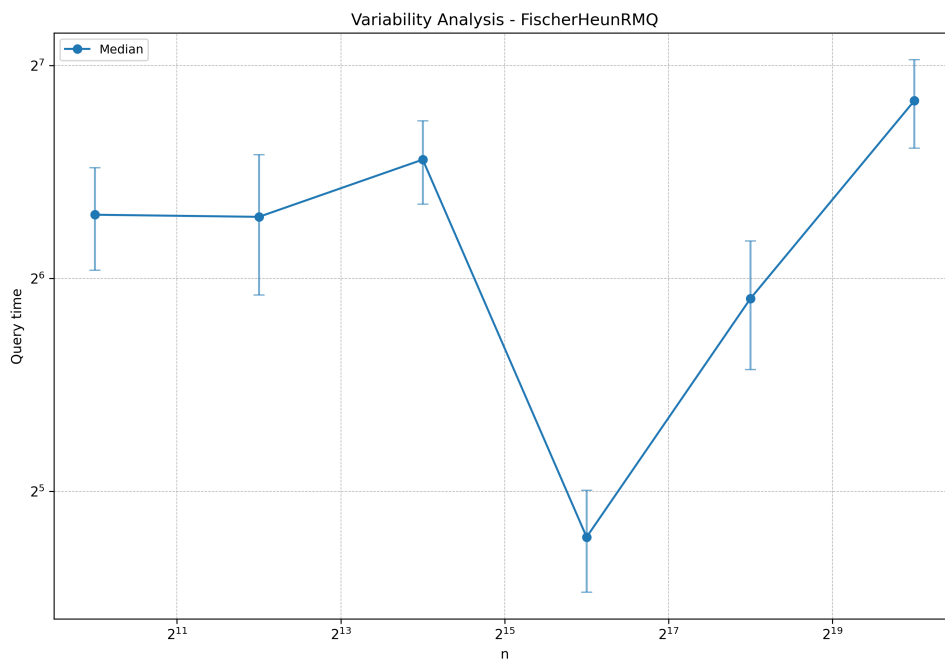


Figura 5: Variabilidad del tiempo de consulta para **Fischer-Heun**.

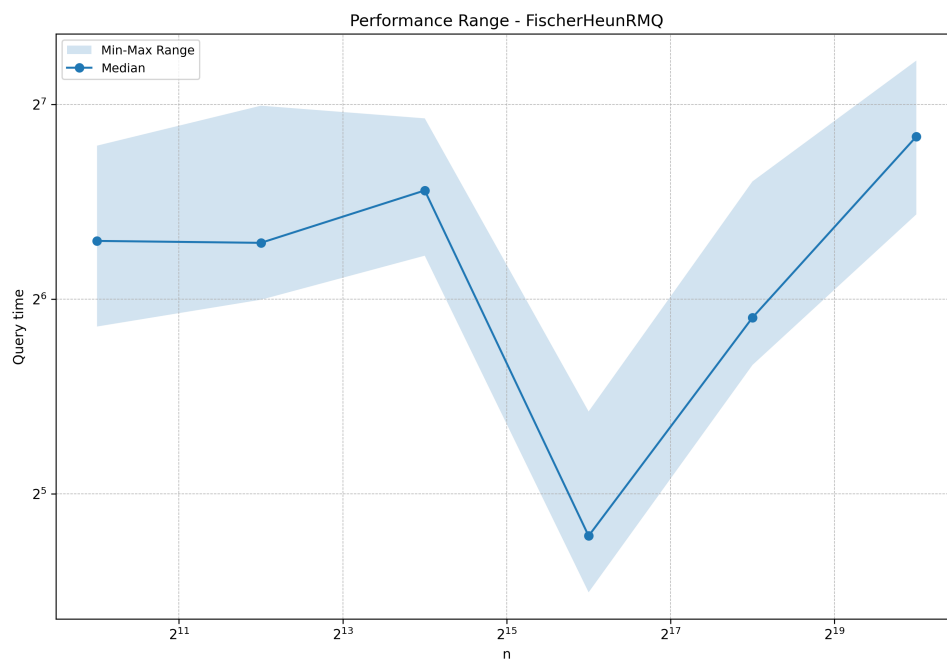


Figura 6: Rango máximo y mínimo del tiempo de consulta para Fischer-Heun.

De manera similar, **Hybrid A** presenta un comportamiento muy estable para tamaños pequeños y medianos, aunque para tamaños superiores a 2^{14} comienzan a aparecer diferencias más visibles entre media y mediana, una mayor desviación estándar y máximos más elevados, indicando la presencia de algunos valores atípicos.

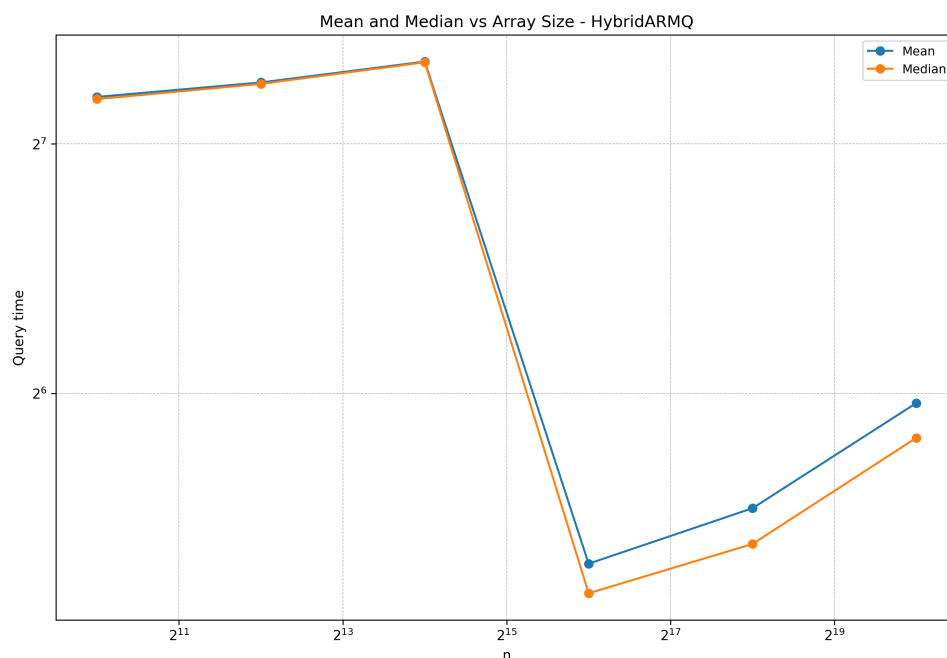


Figura 7: Media y mediana del tiempo de consulta para Hybrid A.

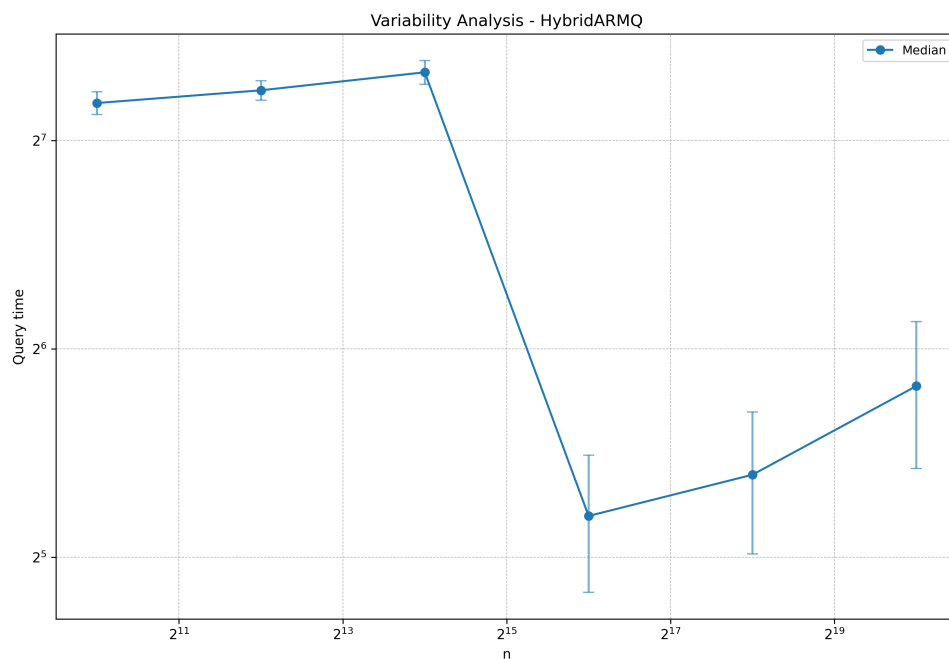


Figura 8: Variabilidad del tiempo de consulta para Hybrid A.

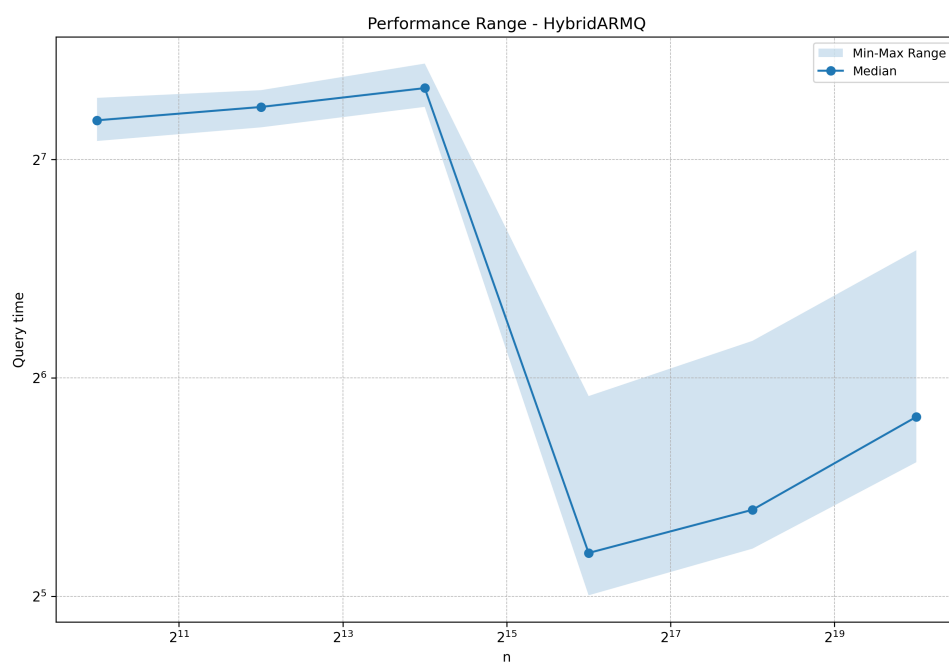


Figura 9: Rango máximo y mínimo del tiempo de consulta para Hybrid A .

Hybrid B muestra una variabilidad superior a la observada en Hybrid A, con máximos más alejados de la mediana y una desviación estándar relativamente elevada.

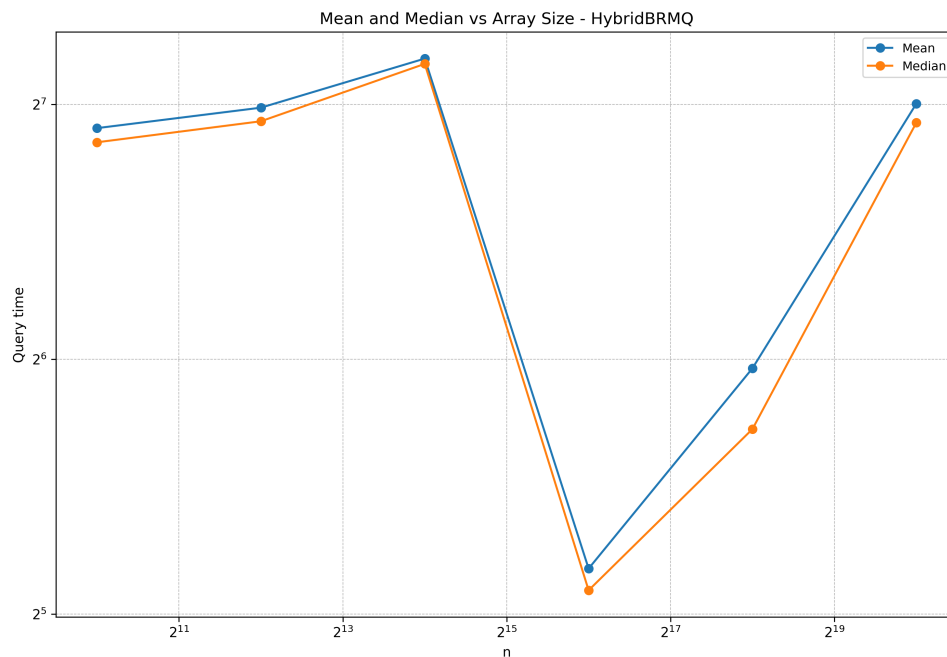


Figura 10: Media y mediana del tiempo de consulta para Hybrid B.

Por otro lado, **Naive RMQ**, **Block Decomposition** y **Segment Tree** presentan diferencias muy reducidas entre media y mediana, así como desviaciones estándar pequeñas, lo que sugiere un comportamiento particularmente estable a través de las distintas ejecuciones.

Mientras que **Hybrid C** posee cualidades estables en valores pequeños de tamaño y una variabilidad alta en tamaños medios y grandes. Además **Full Preprocessing** en los datos que se tomaron tiene una variabilidad que va disminuyendo a medida que se aumenta la cantidad de datos.

4.5. Throughput

La gráfica de throughput permite observar cuántas consultas por segundo puede procesar cada estructura de datos.

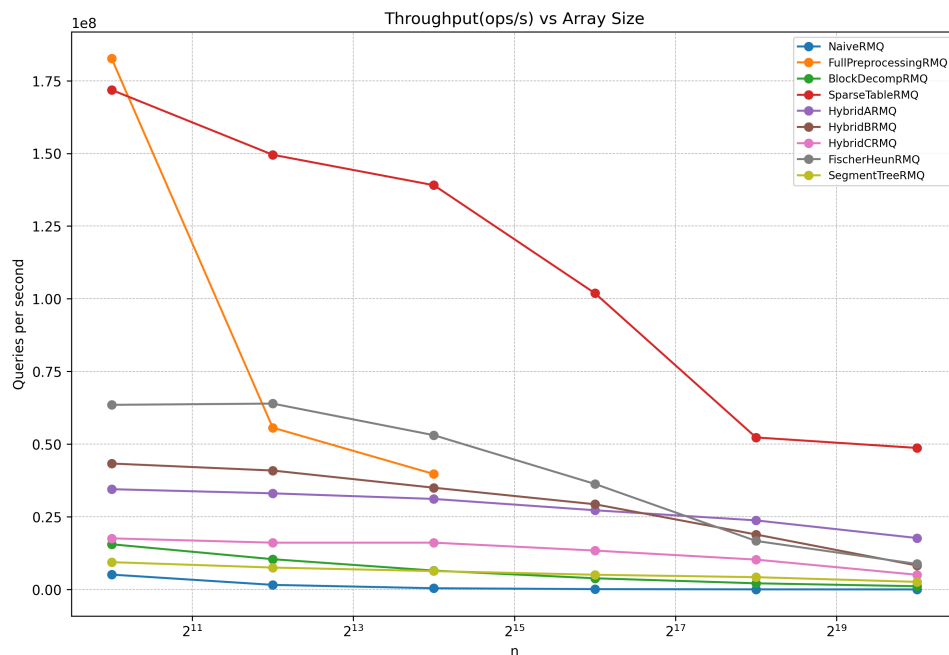


Figura 11: Throughput de las estructuras en función del tamaño del arreglo.

Como era de esperar, las estructuras con menores tiempos de consulta presentan también los mayores valores de throughput. **Sparse Table** obtiene consistentemente el mejor rendimiento, seguida por **Hybrid A**, que vuelve a destacar por ofrecer un excelente compromiso entre simplicidad y eficiencia práctica.

Las estructuras **Fischer-Heun** y **Hybrid B** presentan un throughput competitivo, aunque inferior al de **Sparse Table**. Por otro lado, **Segment Tree**, **Hybrid C** y **Block Decomposition** muestran valores intermedios, consistentes con sus complejidades teóricas de consulta.

Finalmente, **Naive RMQ** presenta el menor throughput de todas las estructuras debido a su costo lineal por consulta. A medida que aumenta el tamaño del arreglo, la cantidad de operaciones que puede procesar por segundo disminuye significativamente.

4.6. Análisis general

Las Figuras 12 y 13 resumen simultáneamente tres de las métricas más importantes del estudio: tiempo de preprocesamiento, tiempo de consulta y consumo de memoria. Cada punto representa una estructura de datos evaluada utilizando el mayor tamaño de entrada considerado, mientras que el tamaño de la burbuja refleja el uso de memoria asociado.

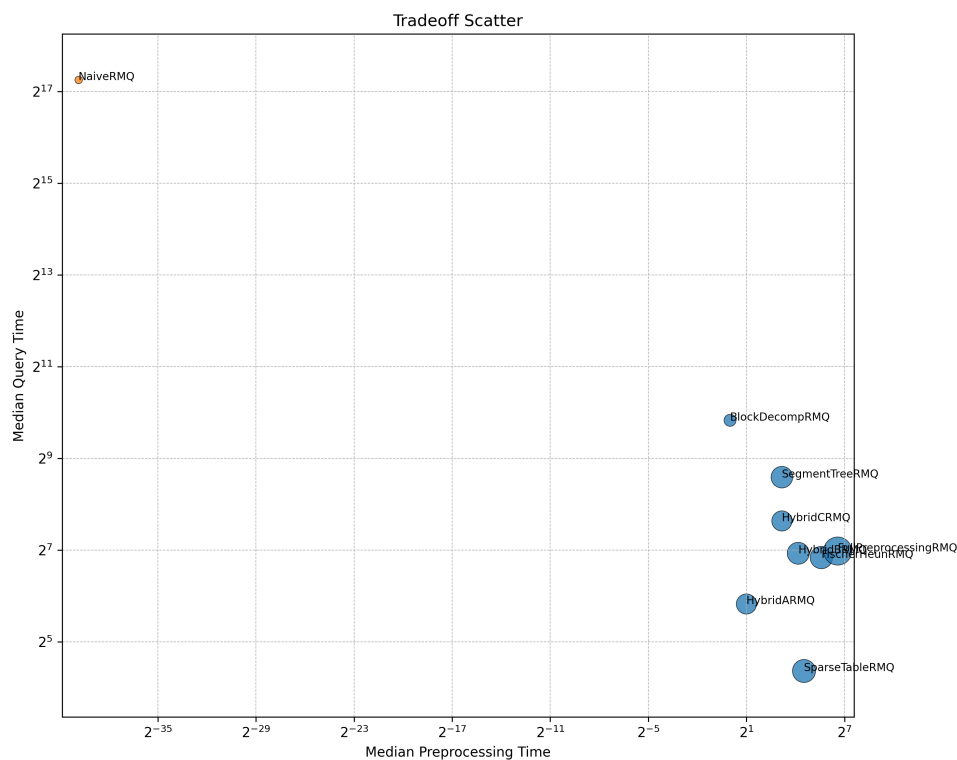


Figura 12: Relación entre tiempo de procesamiento, tiempo de consulta y uso de memoria para las distintas estructuras de datos.

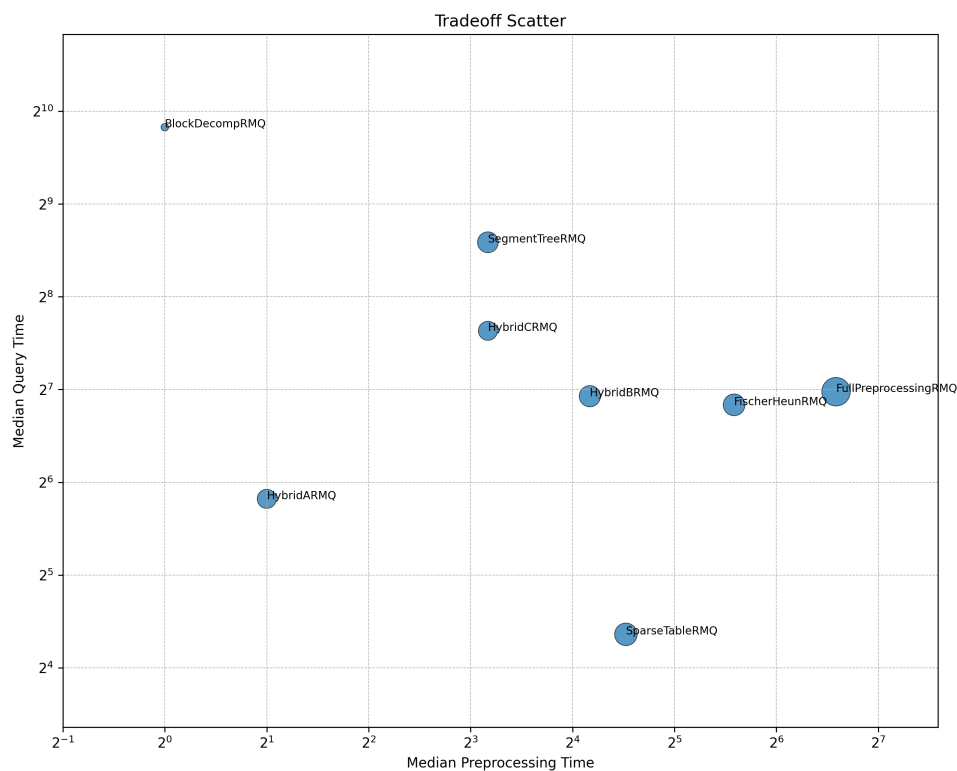


Figura 13: Relación entre tiempo de procesamiento, tiempo de consulta y uso de memoria excluyendo Naive RMQ para facilitar la visualización.

Complementariamente, la Figura 11 muestra la cantidad de consultas por segundo que cada estructura es capaz de procesar a medida que aumenta el tamaño de entrada.

A partir de estos resultados puede observarse claramente el compromiso existente entre costo de construcción, memoria utilizada y velocidad de consulta. Ninguna estructura domina completamente a las demás en todos los aspectos, por lo que la elección depende de los requerimientos específicos de la aplicación.

Considerando el balance global entre las tres métricas principales, **Hybrid A** emerge como una de las alternativas más atractivas. Su tiempo de preprocesamiento permanece extremadamente bajo incluso para tamaños grandes, su consumo de memoria se mantiene cercano al lineal y sus tiempos de consulta resultan sorprendentemente competitivos. Aunque teóricamente sus consultas tienen complejidad $O(\log n)$, experimentalmente logra rendimientos comparables e incluso superiores a algunas estructuras con complejidad teórica $O(1)$.

Este resultado constituye uno de los hallazgos más interesantes del experimento. La teoría asintótica proporciona una excelente aproximación del comportamiento para tamaños muy grandes, pero en implementaciones reales intervienen factores adicionales como la localidad de memoria, el comportamiento de la caché, el tamaño efectivo de las estructuras auxiliares, la cantidad de accesos indirectos y las constantes ocultas. Como consecuencia, una estructura con complejidad $O(\log n)$ puede superar en la práctica a otra con complejidad $O(1)$.

El mejor ejemplo de este fenómeno se observa al comparar **Hybrid A** con **Fischer-Heun**. Aunque **Fischer-Heun** posee cotas teóricamente óptimas para RMQ, sus tiempos experimentales de consulta no logran superar consistentemente a **Hybrid A**. De manera similar, aunque **Sparse Table** sí consigue posicionarse como la estructura más rápida en consultas, la diferencia respecto a **Hybrid A** resulta mucho menor de lo que podría sugerir únicamente el análisis asintótico.

La gráfica de throughput confirma estas observaciones. **Sparse Table** alcanza sistemáticamente la mayor cantidad de consultas por segundo, consolidándose como la implementación más eficiente durante la fase de consultas. Muy cerca aparece **Hybrid A**, que obtiene un rendimiento sobresaliente pese a poseer una complejidad teórica inferior. Posteriormente se encuentran **Fischer-Heun** y **Hybrid B**, mientras que **Segment Tree**, **Hybrid C** y **Block Decomposition** presentan un throughput considerablemente menor. Finalmente, **Naive RMQ** queda ampliamente rezagado debido al costo lineal de cada consulta.

Resulta interesante notar que incluso después de reducir el número total de consultas ejecutadas para tamaños superiores a 2^{14} , la mayoría de las estructuras muestran una disminución gradual en el throughput conforme aumenta el tamaño del arreglo. Este comportamiento es consistente con el incremento en accesos a memoria, mayores presiones sobre la caché y el crecimiento de las estructuras auxiliares.

Los análisis de estabilidad y variabilidad también permiten complementar la interpretación de los resultados. En términos generales, todas las estructuras presentan diferencias reducidas entre media y mediana, lo que indica una buena consistencia experimental. Sin embargo, algunas implementaciones muestran comportamientos particulares.

Naive RMQ, **Block Decomposition** y **Segment Tree** destacan por ser las estructuras más estables del conjunto. Sus medias y medianas prácticamente coinciden para todos los tamaños evaluados, sus desviaciones estándar permanecen bajas y los rangos entre valores mínimos y máximos son reducidos. Esto sugiere que sus tiempos de ejecución son altamente predecibles.

Por el contrario, **Fischer-Heun**, **Hybrid A**, **Hybrid B** y **Sparse Table** presentan medias consistentemente superiores a sus medianas para algunos tamaños de entrada, indicando la aparición

ocasional de ejecuciones más lentas. Este comportamiento es especialmente visible en **Sparse Table** y **Hybrid B**, donde las desviaciones estándar y los máximos observados se encuentran más alejados de los valores típicos.

En el caso de **Hybrid A**, la estabilidad es excelente para tamaños pequeños y medianos, aunque a partir de 2^{14} comienzan a aparecer máximos más elevados y una variabilidad moderadamente superior. Aun así, el rendimiento típico continúa siendo muy consistente, como demuestra la cercanía entre media y mediana.

Fischer-Heun muestra una situación similar. Aunque mantiene consultas rápidas y relativamente estables, presenta una dispersión mayor que otras estructuras comparables, posiblemente debido a la complejidad de su implementación y al número de estructuras auxiliares involucradas durante la consulta.

Por su parte, **Hybrid C** exhibe un comportamiento estable para tamaños pequeños, pero una variabilidad más visible conforme aumenta el tamaño de entrada. **Full Preprocessing**, en cambio, muestra una tendencia opuesta: su variabilidad disminuye progresivamente a medida que crece el tamaño del arreglo, probablemente porque el costo dominante termina siendo el acceso a una tabla extremadamente grande y uniforme.

Respecto a **Fischer-Heun**, los resultados obtenidos son razonablemente consistentes con sus propiedades teóricas. La estructura alcanza consultas en tiempo constante utilizando espacio lineal, pero las constantes ocultas derivadas de su construcción y de la complejidad interna de la implementación reducen parte de la ventaja que cabría esperar desde un punto de vista puramente teórico. Aun así, representa una demostración práctica de que es posible alcanzar simultáneamente las cotas óptimas para el problema RMQ.

Finalmente, merece una mención especial **Segment Tree**. Aunque no lidera ninguna de las métricas evaluadas, mantiene un desempeño equilibrado, una estabilidad sobresaliente y una implementación relativamente sencilla. Además, su verdadera fortaleza radica en su versatilidad: puede adaptarse fácilmente para soportar actualizaciones dinámicas, mantener distintos tipos de información agregada y resolver consultas considerablemente más complejas que RMQ. Esta flexibilidad explica por qué continúa siendo una de las estructuras más utilizadas en aplicaciones reales a pesar de no ser la más eficiente para consultas RMQ estáticas.